

# CONHECIMENTOS ESPECÍFICOS

## Testes de Software



**Presidente:** Gabriel Granjeiro

**Vice-Presidente:** Rodrigo Calado

**Diretor Pedagógico:** Erico Teixeira

**Diretora de Produção Educacional:** Vivian Higashi

**Gerente de Produção Digital:** Bárbara Guerra

**Coordenadora Pedagógica:** Élica Lopes

Todo o material desta apostila (incluindo textos e imagens) está protegido por direitos autorais do Gran. Será proibida toda forma de plágio, cópia, reprodução ou qualquer outra forma de uso, não autorizada expressamente, seja ela onerosa ou não, sujeitando-se o transgressor às penalidades previstas civil e criminalmente.

**CÓDIGO:**

250205556647



**PATRÍCIA QUINTÃO**

Mestre em Engenharia de Sistemas e computação pela COPPE/UFRJ, Especialista em Gerência de Informática e Bacharel em Informática pela UFV. Atualmente é professora no Gran Cursos Online; Analista Legislativo (Área de Governança de TI), na Assembleia Legislativa de MG; Escritora e Personal & Professional Coach. Atua como professora de Cursinhos e Faculdades, na área de Tecnologia da Informação, desde 2008. É membro: da Sociedade Brasileira de Coaching, do PMI, da ISACA, da Comissão de Estudo de Técnicas de Segurança (CE-21:027.00) da ABNT, responsável pela elaboração das normas brasileiras sobre gestão da Segurança da Informação. Autora dos livros: Informática FCC – Questões comentadas e organizadas por assunto, 3ª. edição e 1001 questões comentadas de informática (Cespe/UnB), 2ª. edição, pela Editora Gen/Método. Foi aprovada nos seguintes concursos: Analista Legislativo, na especialidade de Administração de Rede, na Assembleia Legislativa do Estado de MG; Professora titular do Departamento de Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia; Professora substituta do DCC da UFJF; Analista de TI/Suporte, PRODABEL; Analista do Ministério Público MG; Analista de Sistemas, DATAPREV, Segurança da Informação; Analista de Sistemas, INFRAERO; Analista – TIC, PRODEMGE; Analista de Sistemas, Prefeitura de Juiz de Fora; Analista de Sistemas, SERPRO; Analista Judiciário (Informática), TRF 2ª Região RJ/ES, etc.

**GRAN**  
CONCURSOS

# SUMÁRIO

|                                                             |           |
|-------------------------------------------------------------|-----------|
| Apresentação .....                                          | 4         |
| <b>Testes de Software .....</b>                             | <b>5</b>  |
| Visão Geral sobre os Testes de Software.....                | 5         |
| Verificação e Validação .....                               | 5         |
| Defeitos, Erros e Falhas.....                               | 5         |
| Defeitos no Desenvolvimento de Software .....               | 8         |
| Principais Níveis de Teste de Software (Quando Testar)..... | 10        |
| Técnicas de Teste de Software (Como Testar) .....           | 15        |
| Tipos de Teste de Software (O Que Testar).....              | 17        |
| Exemplos de Artefatos Produzidos no Processo de Teste.....  | 18        |
| <b>Resumo .....</b>                                         | <b>19</b> |
| <b>Questões Comentadas em Aula .....</b>                    | <b>21</b> |
| <b>Questões de Concurso.....</b>                            | <b>22</b> |
| <b>Gabarito .....</b>                                       | <b>28</b> |
| <b>Gabarito Comentado.....</b>                              | <b>29</b> |
| <b>Referências .....</b>                                    | <b>43</b> |

## APRESENTAÇÃO

Olá, querido(a) amigo(a), tudo bem?



Quer conquistar a sua vitória? Então, tenha uma postura grandiosa! Todos que venceram grandes desafios se alimentavam com pensamentos **positivos, engrandecedores**, de **esperança** e de **coragem**.

Mesmo diante das adversidades, conserve a elegância e uma autoimagem positiva. Essa autoimagem é como se fosse um combustível que impulsionará as suas ações.  
#FicaaDica #OSegredo

Rumo então à aula sobre **Testes de Software**.

Em caso de dúvidas, acesse o fórum do curso ou entre em contato.

Um forte abraço!

# TESTES DE SOFTWARE

## VISÃO GERAL SOBRE OS TESTES DE SOFTWARE

Antes de iniciarmos uma discussão sobre **teste de software** precisamos esclarecer alguns conceitos relacionados a essa atividade.

### VERIFICAÇÃO E VALIDAÇÃO

Para alguns estamos diante de conceitos similares: **verificação** e **validação**, porém dentro do contexto de software são coisas distintas que se complementam a fim de garantir a **qualidade do software**.

- **Verificação** é analisar se a forma com a qual estamos construindo um produto (software) é a correta; e
- **Validação** é analisar se o produto (software) é o correto.

As perguntas que diferenciam verificação de validação, são:

1. Verificação: **será que estamos construindo o produto corretamente.**
2. Validação: **será que estamos construindo o produto correto.**

Podemos então concluir que a **verificação tem o objetivo de analisar a especificação do software**, enquanto a **validação** tem o objetivo de analisar se o que o cliente solicitou é o que de fato vai receber.

Na verificação temos então uma análise principalmente da **especificação dos requisitos funcionais e não funcionais** e na validação **temos a expectativa do cliente em relação ao software sendo analisada**. Podemos ter verificação tanto estática como dinâmica.

| Verificação     | Objetivo                                                                                                                                                                                                      |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Estática</b> | Na <b>verificação estática</b> , tem-se uma <b>inspeção do software</b> com a análise de documentação de requisitos, projeto, análise de código etc.                                                          |
| <b>Dinâmica</b> | Na <b>verificação dinâmica</b> , tem-se os famosos <b>testes</b> , ou seja, na verificação dinâmica são analisados os dados de entrada e saída, e verifica-se se o resultado obtido está conforme o esperado. |

Nenhum dos dois tipos de verificação, garante que não haja erros ou defeitos no software em todos os casos possíveis, ou seja: **nada aqui é 100% de garantia.**

### DEFEITOS, ERROS E FALHAS

Vamos à diferença entre **Defeitos**, **Erros** e **Falhas**, conforme visto na figura seguinte.

- **Defeito** é um **ato inconsistente cometido por um indivíduo** ao tentar entender uma determinada informação, resolver um problema ou utilizar um método requisitos ou uma ferramenta. Por exemplo, uma instrução ou comando incorreto;
- **Erro** é uma **manifestação concreta de um defeito num artefato de software**. Diferença entre o valor obtido e o valor esperado, ou seja, qualquer estado intermediário incorreto ou resultado inesperado na execução de um programa constitui um erro;
- **Falha** é o **comportamento operacional do software diferente do esperado pelo usuário**. Uma falha pode ter sido causada por diversos erros e alguns erros podem nunca causar uma falha.

### Defeito, erro e falha



Observe pela figura que **defeitos** fazem parte do universo físico (a aplicação propriamente dita) e são causados por pessoas, por exemplo, através do mal uso de uma tecnologia. **Defeitos** podem ocasionar a manifestação de erros em um produto, ou seja, a construção de um software de forma diferente ao que foi especificado (universo de informação).

Por fim, **os erros geram falhas**, que são comportamentos inesperados em um software que afetam diretamente o usuário final da aplicação (universo do usuário) e pode inviabilizar a utilização de um software.

Dessa forma, ressaltamos que **teste de software revela simplesmente falhas em um produto**.

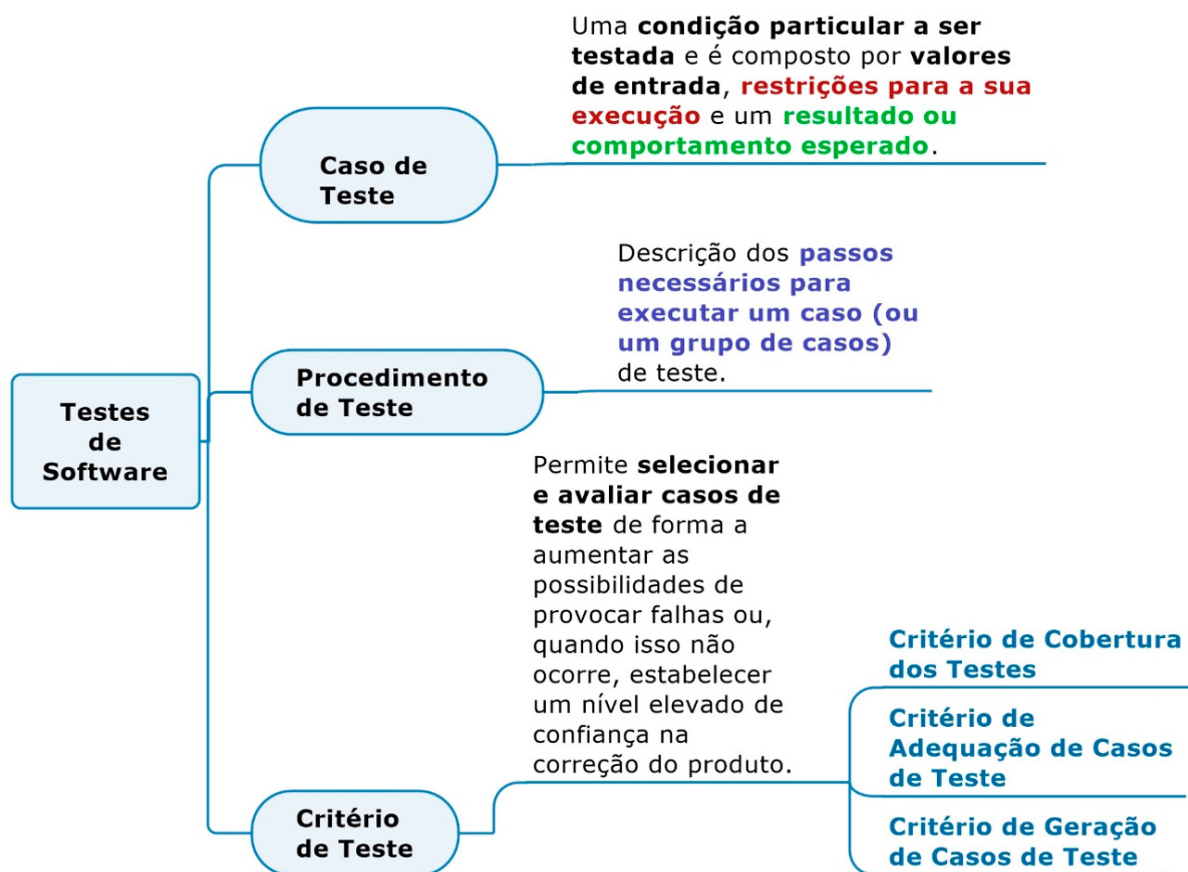
Após a execução dos testes é necessária a execução de um processo de **depuração** para a **identificação** e **correção** dos **defeitos** que originaram essa **falha**, ou seja, **depurar não é testar!**

**Falhas são esperadas nos testes**, ou seja, quando testamos é normal que tenhamos comportamentos incorretos, identificando estes, executamos os ajustes necessários.

Antes de continuar, gostaria de reforçar o entendimento de que **o teste NÃO é uma fase que deve ser executada ao final do projeto, mas uma atividade que deve ser exercida ao longo do processo de desenvolvimento do software**.

A **atividade de teste** é composta por alguns elementos essenciais que auxiliam na formalização desta atividade. São eles:

#### Testes de software



Fonte: Elaboração própria (2022).

Os **critérios de teste** podem ser utilizados como:

|                                         |                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critério de Cobertura dos Testes        | <p><b>Permitem</b> a identificação de partes do programa <b>que devem ser executadas para garantir a qualidade do software e indicar quando o mesmo foi suficientemente testado</b>.</p> <p><b>Assim, busca determinar o percentual de elementos necessários por um critério de teste que foram executados pelo conjunto de casos de teste.</b></p> |
| Critério de Adequação de Casos de Teste | <p>Quando, a partir de um conjunto de casos de teste T qualquer, ele é utilizado para verificar se T satisfaz os requisitos de teste estabelecidos pelo critério.</p> <p><b>Avalia se os casos de teste definidos são suficientes ou não</b> para avaliação de um produto ou uma função.</p>                                                        |
| Critério de Geração de Casos de Teste   | <p>Quando o critério é utilizado para gerar um conjunto de casos de teste T adequado para um produto ou função, ou seja, este critério <b>define as regras e diretrizes para geração dos casos de teste de um produto que esteja de acordo com o critério de adequação definido anteriormente</b>.</p>                                              |



## DEFEITOS NO DESENVOLVIMENTO DE SOFTWARE

No processo de desenvolvimento de software, todos os defeitos são humanos e, apesar do uso dos melhores métodos de desenvolvimento, ferramentas ou profissionais, permanecem presentes nos produtos, o que torna a **atividade de teste fundamental** durante o desenvolvimento de um software. E essa atividade corresponde ao **último recurso para avaliação do produto antes da sua entrega ao usuário final**.

O tamanho do projeto a ser desenvolvido e a quantidade de pessoas envolvidas no processo são dois possíveis fatores que aumentam a complexidade dessa tarefa, e consequentemente aumentam a probabilidade de defeitos. Assim, a ocorrência de **falhas** é inevitável.

Mas o que significa dizer que um programa **falhou**? Basicamente significa que o **funcionamento do programa não está de acordo com o esperado pelo usuário**. Por exemplo, quando um usuário da linha de produção efetua consultas no sistema das quais só a gerência deveria ter acesso.

Esse tipo de falha pode ser originado por diversos motivos:

- A **especificação pode estar errada** ou incompleta;
- A **especificação pode conter requisitos impossíveis de serem implementados** devido às limitações de hardware ou software;
- A **base de dados pode estar organizada de forma que não seja permitido distinguir os tipos de usuário**;
- Pode ser que haja um **defeito no algoritmo** de controle dos usuários.

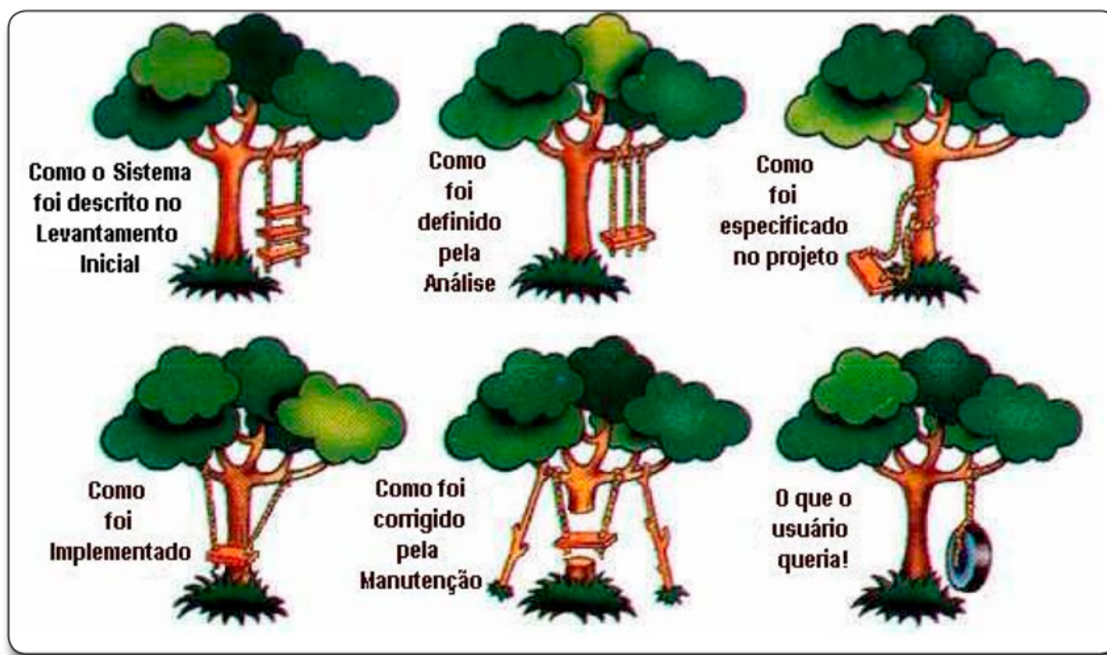
Os **defeitos** normalmente são introduzidos na transformação de informações entre as diferentes fases do ciclo de desenvolvimento de um software.

Vamos seguir um exemplo simples de ciclo de vida de desenvolvimento de software: os **requisitos** expressos pelo cliente são relatados textualmente em um documento de especificação de requisitos. Esse documento é então transformado em **casos de uso**, que por sua vez foi o artefato de entrada para o projeto do software e definição de sua arquitetura utilizando diagramas de classes da UML. Em seguida, esses modelos de projetos foram usados para a construção do software em uma linguagem que não segue o paradigma orientado a objetos.

Observe que durante esse período uma série de transformações foi realizada até chegarmos ao produto final. Nesse meio tempo, defeitos podem ter sido inseridos.



## Diferentes interpretações ao longo do ciclo de desenvolvimento de um software



A figura anterior expressa exatamente a **metáfora** discutida nesse parágrafo. **Essa série de transformações resultou na necessidade de realizar testes em diferentes níveis, visando avaliar o software em diferentes perspectivas de acordo com o produto gerado em cada fase do ciclo de vida de desenvolvimento de um software.**

Quando o **teste** se inicia há um conflito de interesses:

- **desenvolvedores:** interesse em demonstrar que o programa é isento de erros;
- **responsáveis** pelos testes: interesse em mostrar que o programa tem erros.

Diversas atividades de **teste** são realizadas para **verificação do sistema construído**, levando-se em conta a especificação feita na fase de projeto.

O **teste de software** como o nome já diz significa **submeter um software a ser testado, com o objetivo de encontrar anomalias e problemas**, para de fato adequar o software ao que o cliente deseja, verificando assim o trabalho (código) implementado pelo programador.

**Teste de software** é uma **atividade cara que exige tempo, planejamento, conhecimento técnico, infraestrutura e comprometimento**.

Por maior e mais organizado que seja o esforço para a realização das atividades de testes, é **impossível garantir 100% de cobertura de todos os requisitos ou linhas de código existentes no sistema**.

Edsger Dijkstra sabiamente resumiu este fato com a seguinte frase: **“Testes podem mostrar a presença de erros, mas não a sua ausência”**. Dessa forma, está em suas mãos a responsabilidade de conhecer e aplicar as melhores práticas na gestão de testes para garantir a maior cobertura com o mínimo de esforço.

Uma importante razão para testar é identificar e excluir defeitos do software, mas **testar não garante que o software esteja isento de defeitos**.

Temos na literatura a conceituação de **processo de teste** que são as etapas implementadas a fim de **orientar os testes que serão realizados**.

Vamos aprofundar mais nesse assunto, destacando **tipos de testes**, **níveis de teste** e as **técnicas de testes**:

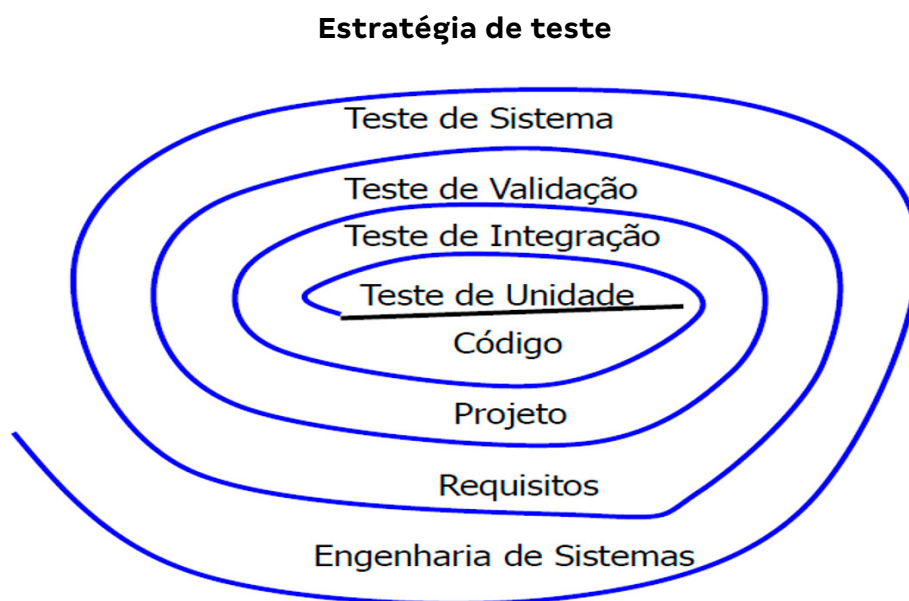
- os **tipos de testes** se referem **ao que vamos testar**;
- os **níveis de testes** se referem ao **momento no qual vamos testar**; e
- as **técnicas de testes** são de fato a **forma** pela qual nós vamos testar.

Veja o quadro seguinte com as três definições:

| Técnica                                                                                         | Nível               | Dimensões da Qualidade                                                                            |                |             |            |                 |
|-------------------------------------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------|----------------|-------------|------------|-----------------|
|                                                                                                 |                     | Funcionalidade                                                                                    | Confiabilidade | Usabilidade | Desempenho | Suportabilidade |
|                                                                                                 |                     | Tipos de Teste (alguns exemplos)                                                                  |                |             |            |                 |
| Caixa Branca                                                                                    | Teste de Unidade    |                                                                                                   |                |             |            |                 |
| Caixa Cinza                                                                                     | Teste de Integração | Segurança                                                                                         | Integridade    |             | Carga      | Configuração    |
| Caixa Preta                                                                                     | Teste de Sistema    | Funcional                                                                                         | Regressão      | Usabilidade |            |                 |
|                                                                                                 | Teste de Aceitação  | Volume                                                                                            | Maturidade     |             | Estresse   | Instalação      |
|  Como Testar |                     |  Quando Testar |                |             |            |                 |
|                                                                                                 |                     |  O que Testar |                |             |            |                 |

## PRINCIPAIS NÍVEIS DE TESTE DE SOFTWARE (QUANDO TESTAR)

Temos quatro **níveis de testes**: **teste de unidade**, **teste de integração**, **teste de validação** e **teste de sistema**, que vão delimitar **QUANDO iremos testar**.



Fonte: Pressman (2011).

Os **níveis de teste** são apresentados na figura conforme o contexto de cada um dos testes. Conforme visto, Pressman (2011) propõe esse **modelo espiral** para demonstrar a estratégia de teste.

- O **teste de unidade** começa no centro da espiral e se concentra em cada unidade (por exemplo: componente, classe, ou objeto de conteúdo de WebApp) do software conforme implementado no código-fonte;
- O teste prossegue movendo-se em direção ao exterior da espiral, passando pelo **teste de integração**, em que o foco está no projeto e construção da arquitetura de software;
- Continuando na mesma direção da espiral, encontramos o **teste de validação**, em que requisitos estabelecidos como parte dos requisitos de modelagem são validados em relação ao software criado;
- Finalmente, chegamos ao **teste do sistema**, no qual o software e outros elementos são testados como um todo. Para testar um software de computador, percorre-se a espiral em direção ao seu exterior, no sentido horário, ao longo de linhas que indicam o escopo do teste a cada volta.

Os principais **níveis de teste de software** são:

#### TESTE DE UNIDADE (TESTES UNITÁRIOS)

O primeiro **tipo de teste** que temos é o **teste de unidade** que trabalha com técnicas de teste para detecção de erros em componentes menores (**unidades**) que serão integradas já testadas.

Tem por objetivo **explorar a menor unidade do projeto**, procurando provocar falhas ocasionadas por defeitos de lógica e de implementação em cada módulo, separadamente.

O universo alvo desse tipo de teste são os métodos dos objetos ou mesmo pequenos trechos de código.

## TESTE DE INTEGRAÇÃO

Tem o objetivo de analisar problemas na **construção (integração) dos componentes dentro de um software**. Visa provocar falhas associadas às interfaces entre os módulos quando esses são integrados para construir a estrutura do software que foi estabelecida na fase de projeto.

O teste de integração pode se basear nas seguintes abordagens convencionais:

### a) Big-Bang

Integração **não** incremental. Todos os componentes são combinados com antecedência. O programa é testado como um todo.

### b) Incremental

O programa é construído e integrado em pequenos incrementos.

- **Descendente (top-down)**: os módulos são integrados deslocando-se para baixo através da hierarquia de controle, começando com o módulo de controle principal. Verifica os principais pontos de controle ou decisão antecipada no processo. Pode ser realizada: (1) primeiro-em-profundidade; (2) primeiro-em-largura;
- **Ascendente (bottom-up)**: começa a construção e o teste com módulos atômicos.

A seguir destacamos algumas abordagens especiais de teste de integração são:

- **Teste de regressão**: **não** corresponde a um nível de teste, mas é uma estratégia importante para redução de “efeitos colaterais”. **Consiste em se aplicar, a cada nova versão do software ou a cada ciclo, todos os testes que já foram aplicados nas versões ou ciclos de teste anteriores do sistema**. Pode ser aplicado em qualquer nível de teste. Pressman (2021) destaca que

o teste de regressão é a reexecução do mesmo subconjunto de testes que já foram executados, para assegurar que as alterações não tenham propagado efeitos colaterais indesejados. **Execute testes de regressão toda vez que for feita uma alteração grande no software** (incluindo a integração de novos componentes). **O teste de regressão ajuda a garantir que as alterações (devido ao teste ou por outras razões) não introduzam comportamento indesejado ou erros adicionais.**

- **Teste fumaça**: séries de testes criada com o objetivo de expor erros que impedem uma construção (*build*) de executar exatamente sua função. A finalidade deverá ser descobrir erros “bloqueadores” que apresentam a mais alta probabilidade de atrasar o cronograma do software. A construção é integrada com outras construções, e o produto inteiro passa diariamente pelo teste fumaça.

## DIRETO DO CONCURSO

**001.** (CESPE/INPI/ANALISTA DE DESENVOLVIMENTO DE SISTEMAS/2013) No teste de integração, verifica-se se o funcionamento em conjunto dos componentes do sistema, se são chamados corretamente e se a transferência de dados acontece no tempo correto, por meio de suas interfaces.



O teste de integração visa provocar **falhas associadas às interfaces entre os módulos** quando esses são integrados para construir a estrutura do software que foi estabelecida na fase de projeto.

**Certo.**

### TESTE DE VALIDAÇÃO

Nesse contexto os **requisitos** estabelecidos como parte dos requisitos de modelagem **são validados** em relação ao software criado.

O **teste de validação** começa quando termina o teste de integração, quando os componentes individuais já foram exercitados, o software está completamente montado como um pacote e os erros de interface já foram descobertos e corrigidos. O teste focaliza ações visíveis ao usuário e saídas do sistema reconhecidas por ele.

Fornece a garantia final de que o **software satisfaz aos requisitos informativos, funcionais, comportamentais e de desempenho**.

Os **testes de validação** incluem:

#### a) Aceitação Informal (Teste Alfa)

O **teste Alfa** é realizado por usuários e **ocorre no ambiente de desenvolvimento**, logo temos um **ambiente controlado** para os usuários realizarem os testes.

Alguns **benefícios** dessa forma de teste são (IBM, 1987):

- As funções e os recursos a serem testados são conhecidos;
- O progresso dos testes pode ser medido e monitorado;
- Os critérios de aceitabilidade são conhecidos;
- Serão revelados defeitos mais subjetivos do que no teste de aceitação formal.

As **desvantagens** incluem (IBM, 1987):

- São necessários recursos, planejamento e recursos de gerenciamento;
- Você não tem controle sobre os casos de teste que são utilizados;
- Os usuários podem se adaptar à forma como o sistema funciona e não encontrar defeitos;



- Os usuários podem se concentrar na comparação do novo sistema com um sistema legado, em vez de procurar defeitos;
- Os recursos do teste de aceitação não estão mais sob o controle do projeto e podem ficar limitados.

### **b) Teste Beta**

O **teste Beta** é realizado pelos usuários também, mas em **ambiente real**, no qual o software será utilizado. O cliente registra todos os problemas encontrados durante o teste e relata esses problemas para o desenvolvedor em intervalos regulares.

O **teste Beta** é o menos controlado das três estratégias de teste de aceitação. No **teste Beta**, a quantidade de detalhes, os dados e a abordagem adotadas são de inteira escolha do testador individual. Cada testador é responsável por criar o próprio ambiente, selecionar os dados correspondentes e determinar as funções, os recursos ou as tarefas a serem exploradas. Cada um deles é responsável por identificar seus próprios critérios para aceitar, ou não, o sistema em seu estado atual.

O **teste Beta** é implementado por usuários, geralmente com pouco ou nenhum gerenciamento por parte da organização de desenvolvimento (ou outra que não seja do usuário final). O **teste Beta** é o mais subjetivo de todas as estratégias de teste de aceitação.

Alguns **benefícios** dessa forma de teste são (IBM, 1987):

- O teste é **implementado por usuários**;
- Há grandes volumes de potenciais recursos de teste;
- Há uma **maior satisfação do cliente para aqueles que participam**;
- **São revelados defeitos mais subjetivos** que o teste de aceitação formal ou informal.

As **desvantagens** incluem (IBM, 1987):

- Talvez você não teste todas as funções ou os recursos;
- **É difícil medir o progresso do teste**;
- Os usuários podem se adaptar à forma como o sistema funciona e não encontrar ou relatar defeitos;
- Os usuários podem se concentrar na comparação do novo sistema com um sistema legado, em vez de procurar defeitos;
- Os recursos do teste de aceitação não estão mais sob o controle do projeto e podem ficar limitados;
- Os critérios de aceitabilidade não são conhecidos;
- São necessários recursos com suporte adicional para gerenciar os testadores beta.

### **c) Teste de Aceitação do Cliente**

Uma **variação do teste beta**, chamada de **teste de aceitação do cliente**, às vezes é executada quando é fornecido software personalizado a um cliente sob contrato.

O cliente executa uma série de testes específicos na tentativa de descobrir erros antes de aceitar o software do desenvolvedor. Em alguns casos (Ex.: Em um grande sistema corporativo ou governamental) o teste de aceitação pode ser muito formal e levar vários dias ou mesmo semanas.

No **teste alfa** temos o usuário utilizando o software em **ambiente controlado** e no **teste beta** temos o usuário utilizando software em ambiente real.

A estratégia a ser selecionada baseia-se geralmente nos requisitos contratuais, nos padrões organizacionais e corporativos, bem como no domínio do aplicativo (IBM, 1987).

## TESTE DE SISTEMA

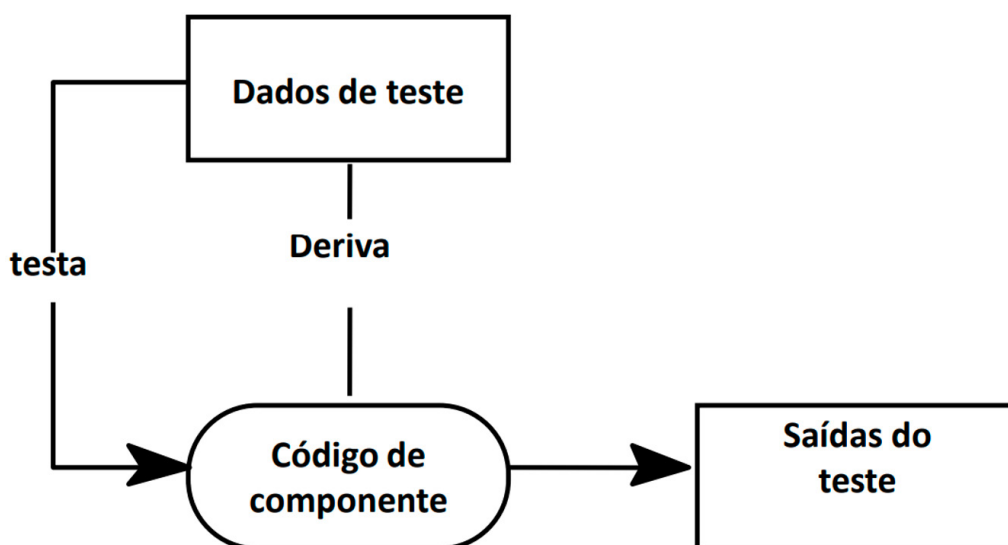
Verifica se os elementos estão coerentes na dimensão de sistema. Avalia o software em busca de **falhas** por meio da utilização do mesmo, como se fosse um usuário final. Dessa maneira, os testes são executados nos mesmos ambientes, com as mesmas condições e com os mesmos dados de entrada que um usuário utilizaria no seu dia a dia de manipulação do software. Verifica se o produto satisfaz seus requisitos.

## TÉCNICAS DE TESTE DE SOFTWARE (COMO TESTAR)

Por fim temos as **técnicas de teste**: **caixa-branca**, **caixa-cinza** e **caixa-preta**.

### TÉCNICA DE CAIXA BRANCA (ESTRUTURAL, WHITE BOX OU CAIXA DE VIDRO)

É responsável por **testar os caminhos lógicos possíveis com foco na parte interna do software**.



Fonte:  
PRESSMAN, ROGER - Engenharia de Software - 6ª Edição  
SOMMERVILLE - Engenharia de Software - 8ª Edição

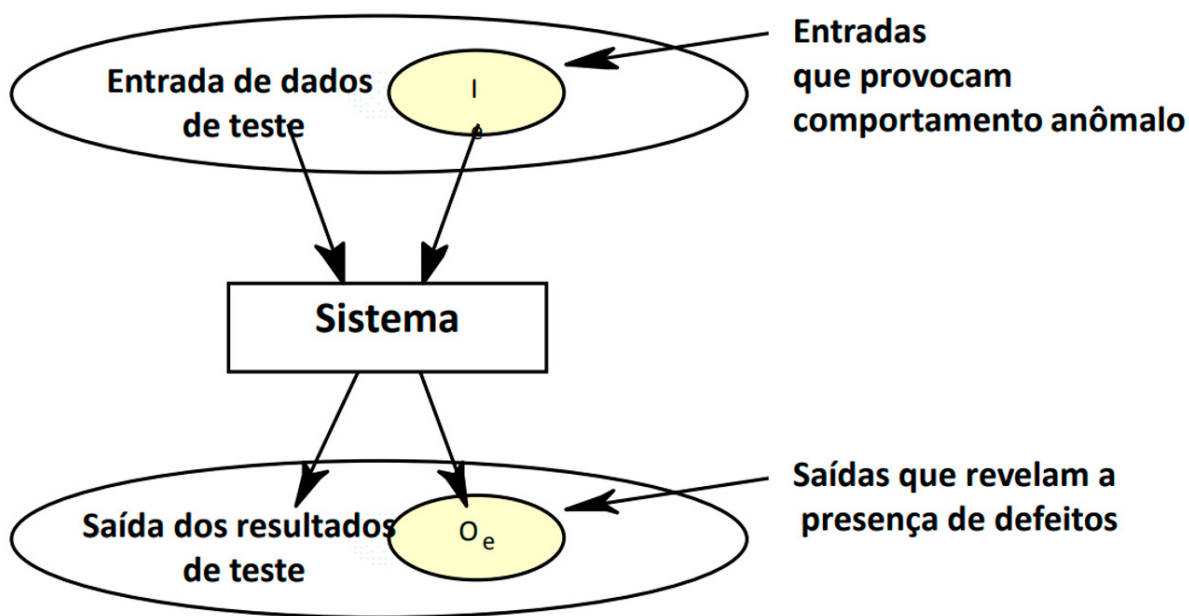


### TÉCNICA DE CAIXA PRETA (FUNCIONAL, COMPORTAMENTAL OU BLACK BOX)

Testa pré-condições avaliando o **comportamento das funcionalidades** conforme os requisitos **sem se preocupar com a lógica interna**.

Envolve dois **passos principais**:

- Identificar as funções que o software deve realizar (especificação dos requisitos);
- Criar casos de teste capazes de checar se essas funções estão sendo executadas corretamente.



### DIRETO DO CONCURSO

**002.** (CESPE/MPU/ANALISTA JUDICIÁRIO/SISTEMAS/2013) Testes funcionais são aplicados para identificar não conformidades entre o programa e seus requisitos.



A técnica de **caixa preta (funcional)** testa pré-condições avaliando o **comportamento das funcionalidades** conforme os requisitos **sem se preocupar com a lógica interna**.

Logo eu estou buscando nos testes funcionais não conformidades entre as funcionalidades do software e os requisitos.

**Certo.**

## TÉCNICA DE CAIXA CINZA

Utiliza **conjuntamente as técnicas de caixa branca e caixa preta**, em que conhecemos a estrutura interna do software e verificamos as estradas e saídas das funcionalidades, exemplo disso é o teste de integração.

## TIPOS DE TESTE DE SOFTWARE (O QUE TESTAR)

Nos **tipos de testes**, estamos preocupados com **O QUE vamos testar**, logo temos vários exemplos dependendo daquilo que desejamos testar (Segurança, Integridade, **teste de carga**, teste de usabilidade, **Desempenho** etc.).

Vamos detalhar alguns desses **tipos** principais:

### TESTE DE DESEMPENHO

Trata-se de um tipo de teste de software que **ajuda a determinar o desempenho de um software** em termos de **velocidade**, **tempo de resposta**, **escalabilidade**, **uso de recursos** e **estabilidade** sob uma determinada carga de trabalho.

#### EXEMPLO

Exemplo de pergunta para esse teste: a aplicação suporta 1.000 transações por minuto com 1.000 usuários simultâneos?

### TESTE DE CARGA

É a prática de **simular o uso do mundo real**, ou carregar, em qualquer software, site, aplicativo web, API ou sistema para analisar e identificar fatores como capacidade de resposta, degradação e escalabilidade.

#### EXEMPLO

Exemplo de pergunta para esse teste: quantas transações serão suportadas por minuto quando aumentarmos os usuários simultâneos para 2.000, 3.000, 4.000?

### TESTE DE STRESS (TESTE POR ESFORÇO)

**Usado para determinar o limite da infraestrutura de TI.** Procura estabelecer até que ponto a infraestrutura, hardware, software etc. podem ser estressados e, a partir daí, fornecer um caderno de recomendações e melhorias.

**Teste por esforço (stress):** usa um sistema de forma que **demande recursos** em quantidade, frequência ou volume **anormal**.

**O stress test na TI ajuda as empresas a saberem quais servidores, software e outras tecnologias precisam de manutenção para aguentar picos de requisições**, bem como definir as tecnologias que devem ser alocadas para garantir o melhor desempenho da sua TI.

## EXEMPLO

Exemplo de pergunta para esse teste: Quantas transações por minuto solicitadas por 5.000, 6.000, 7.000 usuários simultâneos, serão suportadas pela aplicação sob condições não especificadas do software e até mesmo do próprio hardware?

## TESTE DE VULNERABILIDADES

Permitem descobrir quais **vulnerabilidades** estão presentes no ambiente através de revisões abrangentes e sistemáticas, buscando identificar no sistema quaisquer **pontos fracos** que o torne suscetível a ataques ou tentativas de *hacker*.

## TESTE DE SEGURANÇA

- Tenta verificar **se os mecanismos de proteção do sistema irão de fato proteger o sistema contra invasão**;
- Tenta verificar **se todos os mecanismos de proteção embutidos num sistema o protegerão, de fato, de acessos indevidos**;
- Essa estratégia de teste deve ser feita por programadores que **não** desenvolveram o software;
- Os testadores devem tentar invadir o sistema de várias formas: obtenção ilegal de senhas, desarme do sistema etc.

## TESTE DE RECUPERAÇÃO

**Força o software a falhar** de várias formas e verifica se a recuperação é executada corretamente.

## EXEMPLOS DE ARTEFATOS PRODUZIDOS NO PROCESSO DE TESTE

Ao fim do **processo de teste**, podemos ter artefatos como o **plano de testes** e os **casos de testes**.

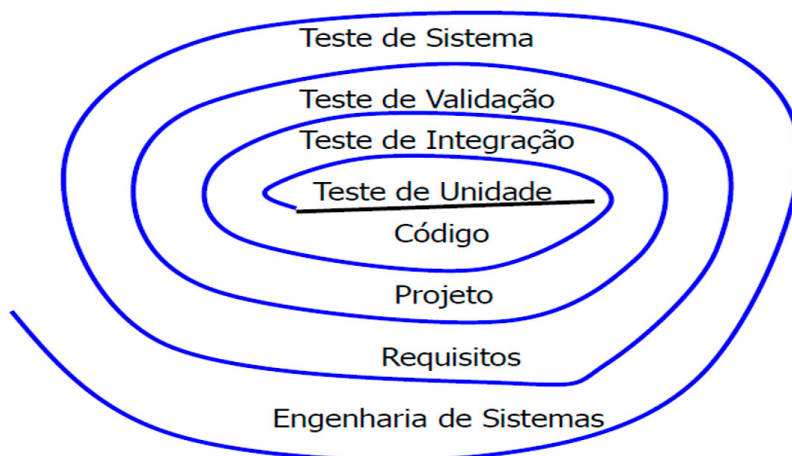
O **plano de testes** representa o **planejamento para execução do teste**, os **casos de testes** apresentam as **funcionalidades testadas**.

O principal **produto** da **fase de Testes** é o **relatório de testes**, que **contém informações sobre erros detectados no software**. Após a atividade de testes, os diversos módulos do sistema são integrados, resultando finalmente no produto de software (BEZERRA, 2007).

## RESUMO

- O **teste de software** é uma das atividades mais custosas do processo de desenvolvimento de software, pois pode envolver uma quantidade significativa dos recursos de um projeto;
- Diferentes tipos de aplicações possuem diferentes técnicas de teste a serem aplicadas, ou seja, testar uma aplicação web envolve passos diferenciados em comparação aos testes de um sistema embarcado;
- Cada tipo de aplicação possui características específicas que devem ser consideradas no momento da realização dos testes;
- **Teste de Unidade**: os pequenos elementos passíveis de teste são testados individualmente, geralmente ao mesmo tempo em que são implementados (KRUCHTEN, 2003);
- **Teste “Caixa-Preta” (Black Box)**: analisa a funcionalidade e a aderência aos requisitos, em uma ótica externa ou do usuário, sem se basear em qualquer conhecimento do código e da lógica interna do componente testado;
- **Teste de Caixa Branca**: é uma abordagem para projetar casos de teste na qual os testes são derivados do conhecimento da estrutura e da implementação do software. O entendimento do algoritmo usado em um componente pode ajudar a identificar partições e casos de teste adicionais (SOMMERVILLE, 2007). O teste “caixa-branca” avalia a lógica interna do sistema;
- O **Técnica de Caixa Cinza** utiliza **conjuntamente as técnicas de caixa branca e caixa preta**, em que conhecemos a estrutura interna do software e verificamos as estradas e saídas das funcionalidades, exemplo disso é o teste de integração;

### Estratégia de teste



Fonte: Pressman (2011).

- **Mock Objects** são objetos simulados que imitam o comportamento de objetos reais, permitindo que os desenvolvedores testem o código de forma isolada, sem depender de outros componentes ou sistemas externos. Trata-se de um recurso utilizado para simular o comportamento de um objeto real. Ele é conhecido, também, como um objeto simulado.

## QUESTÕES COMENTADAS EM AULA

**001.** (CESPE/INPI/ANALISTA DE DESENVOLVIMENTO DE SISTEMAS/2013) No teste de integração, verifica-se se o funcionamento em conjunto dos componentes do sistema, se são chamados corretamente e se a transferência de dados acontece no tempo correto, por meio de suas interfaces.

**002.** (CESPE/MPU/ANALISTA JUDICIÁRIO/SISTEMAS/2013) Testes funcionais são aplicados para identificar não conformidades entre o programa e seus requisitos.



## QUESTÕES DE CONCURSO

**003.** (QUADRIX/CFO/ANALISTA/DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escolha da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

O teste de caixa branca concentra-se em testar a lógica interna do código e os fluxos de controle, garantindo que todas as possíveis condições e os caminhos do código sejam executados para verificar a cobertura de código e a precisão dos algoritmos.

**004.** (QUADRIX/CFO/ANALISTA/DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escolha da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

Os testes de loops são importantes para verificar se os loops no código estão funcionando corretamente, testando tanto as condições de entrada quanto as de saída, e assegurando que o loop não caia em um loop infinito ou produza resultados incorretos.

**005.** (CEBRASPE/CESPE/TRF 6ª REGIÃO/TÉCNICO JUDICIÁRIO/DESENVOLVIMENTO DE SISTEMAS DE INFORMAÇÃO/2025) A respeito de aplicação de testes, julgue o item a seguir. Para facilitar os testes de uma aplicação, podem ser utilizados os mock objects, que são objetos genéricos que atendem a todas as necessidades de testes.

**006.** (QUADRIX/CFO/ANALISTA/DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escolha da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

O teste de caixa preta é eficaz para verificar erros de implementação ou falhas lógicas no código, pois examina os processos internos do sistema e a estrutura do código.

**007.** (INQC/CPTRANS/PROGRAMADOR/2024) Quando se deseja apurar erros de interface durante os casos de teste de software, como por exemplo, o mau uso da interface, utiliza-se o teste:

- a) de componente
- b) de sistema
- c) de release
- d) unitário

**008.** (CEBRASPE/CESPE/CAU BR/ANALISTA/INFRAESTRUTURA DE TECNOLOGIA DA INFORMAÇÃO/2024) Julgue o próximo item, relativo a testes de software.

Considerando que o teste funcional objetiva determinar se um recurso funciona corretamente sem problemas, é possível automatizar esse tipo de teste mesmo que o sistema seja web, em que é possível simular os retornos esperados.

**009.** (CEBRASPE/CESPE/CAU BR/ANALISTA/INFRAESTRUTURA DE TECNOLOGIA DA INFORMAÇÃO/2024) Julgue o próximo item, relativo a testes de software.

Considerada uma técnica sistemática para construir a arquitetura de software concomitantemente à realização de testes para descobrir erros associados às interfaces, o teste de integração realiza testes a partir de componentes testados em unidade.

**010.** (CESGRANRIO/IPEA/TÉCNICO DE PLANEJAMENTO E PESQUISA/DESENVOLVIMENTO DE SISTEMA/2024) Em projetos de desenvolvimento de softwares, é importante que eles sejam testados em diferentes situações comuns de ocorrerem, de modo a contribuir para que eles atinjam a qualidade esperada. Por isso, nesses projetos, quatro níveis de teste são importantes de serem considerados, a fim de ajudar a lidar com tais situações.

Esses quatro níveis de teste são os seguintes:

- a) acessibilidade, usabilidade, unidade e sistema
- b) desempenho, funcionais, componente e aceitação
- c) funcionais, não funcionais, componente e desempenho
- d) funcionais, não funcionais, usabilidade e acessibilidade
- e) unidade, integração, sistema e aceitação

**011.** (CEBRASPE/CESPE/CAU BR/ANALISTA/SISTEMA DE INFORMAÇÃO/2024) Quanto aos princípios fundamentais das atividades de teste e aos processos ágeis de desenvolvimento de software, julgue o item a seguir.

Os estágios das atividades de teste de software devem ser realizados na seguinte ordem: teste do sistema, teste de integração e teste de unidade.

**012.** (CEBRASPE/CESPE/CAU BR/ANALISTA/SISTEMA DE INFORMAÇÃO/2024) Quanto aos princípios fundamentais das atividades de teste e aos processos ágeis de desenvolvimento de software, julgue o item a seguir.

Os testes de regressão são realizados por ocasião da ocorrência de mudanças no software.

**013.** (CEBRASPE/CESPE/INPI/ANALISTA DE PLANEJAMENTO, GESTÃO E INFRAESTRUTURA EM PROPRIEDADE INDUSTRIAL/2024) Julgue o item que se segue, a respeito de qualidade de software.

Um teste de software de regressão estará corretamente projetado quando se considera, em cada uma das funções principais do software, apenas os testes que tratam de uma ou mais classes de erros.

**014.** (INSTITUTO AOCP/CÂMARA DE TERESINA-PI/ANALISTA DE INFORMÁTICA/2021) Cleiton está realizando testes em um sistema e revisa o código procurando por problemas. Nesse caso, ele está realizando testes de qual tipo?

- a) Unitário.
- b) Integração.
- c) Caixa Branca.
- d) Caixa Preta.
- e) A. B.

**015.** (INSTITUTO AOCP/PREFEITURA DE NOVO HAMBURGO-RS/TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS/2020) Qual das seguintes alternativas apresenta uma característica de um teste de caixa preta?

- a) Ignora os mecanismos internos de um sistema e focaliza as saídas geradas em resposta às entradas e condições de execução selecionadas.
- b) É um sinônimo de teste de caixa transparente.
- c) Leva em consideração os mecanismos internos de um sistema componente. É semelhante a um teste estrutural.
- d) Nesse tipo de teste, um exemplo de resultado é a quantidade de erros de acesso da aplicação ao banco de dados.
- e) Um teste de caixa preta é utilizado para determinar se todas as bibliotecas necessárias para a execução do sistema como um todo estão disponíveis para a aplicação.

**016.** (INSTITUTO AOCP/PREFEITURA DE NOVO HAMBURGO-RS/TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS/2020) Assinale a alternativa que apresenta o objetivo de um teste funcional.

- a) Avaliar os aspectos de integridade do banco de dados.
- b) Verificar requisitos de desempenho, como o número de terminais suportados.
- c) Determinar os requisitos de guarda e retenção de dados.
- d) Mapear os aspectos de instabilidade do sistema.
- e) Verificar a consistência entre o produto implementado e respectivos requisitos.

**017.** (CESPE/CEBRASPE/SERPRO/ANALISTA/ESPECIALIZAÇÃO DESENVOLVIMENTO DE SISTEMAS/2021) Acerca dos fundamentos e dos princípios da qualidade de software e da gestão da configuração, julgue o item que se segue.

Realizado o teste unitário de um módulo, o teste de integração contribuirá para a avaliação da existência de erros associados às interfaces do sistema.

**018.** (CESPE/TJ-PA/ANALISTA JUDICIÁRIO/ANÁLISE DE SISTEMAS/2020) No processo de teste de software, o beta teste é

- a) realizado por uma equipe de teste independente.
- b) realizado pelos clientes no local de trabalho do desenvolvedor de software.
- c) realizado por clientes em seu próprio local de trabalho.
- d) útil para testar software sob medida.
- e) executado o mais cedo possível no ciclo de vida.

**019.** (FGV/BANESTES/ANALISTA EM TECNOLOGIA DA INFORMAÇÃO/SUORTE E INFRAESTRUTURA/2018) No contexto de teste de software, o termo “Beta teste” caracteriza testes que:

- a) empregam primordialmente técnicas conhecidas como “White box”;
- b) são equivalentes aos testes conhecidos pelo termo “Alfa teste”;
- c) focam em pontos críticos, cujas correções são providenciadas de imediato pelos desenvolvedores;
- d) são realizados num ambiente de laboratório do desenvolvedor;
- e) são realizados por usuários externos, em condições de uso semelhantes às de produção.

**020.** (CESPE/BNB/ESPECIALISTA TÉCNICO/ANALISTA DE SISTEMA/2018) O item a seguir apresenta uma situação hipotética, seguida de uma assertiva a ser julgada, com relação a testes de software.

Uma equipe de desenvolvimento de softwares pretendia realizar testes de forma incremental durante o desenvolvimento de um programa, a fim de verificar se mudanças no programa não haviam nele introduzido novos bugs; para isso, foram sugeridos os testes unitários e de regressão. Nessa situação, será correto utilizar os testes unitários, mas não os testes de regressão, pois esses últimos não visam verificar novos bugs, mas sim, tão somente, avaliar as funcionalidades do sistema.

**021.** (FGV/DPE-RJ/TÉCNICO SUPERIOR ESPECIALIZADO/DESENVOLVIMENTO DE SISTEMAS/2014) Testes unitários são amplamente empregados no desenvolvimento de software. Sua função principal é:

- a) testar o desempenho do software e de seus componentes.
- b) testar o menor bloco de software desenvolvido, avaliando os resultados obtidos com entradas de dados pré-definidos.
- c) avaliar o comportamento do sistema como um todo e como a integração dos componentes de software se comportam.
- d) avaliar a adequação do software desenvolvido com os requisitos unitários estruturais definidos pelos usuários.
- e) testar os cenários alternativos dos casos de uso, garantindo o comportamento esperado para o sistema.

**022.** (CESPE/MPE-PI/ANALISTA DE INFORMÁTICA/2012) Em teste funcional, o conjunto de valores de entrada válidos pode ser reduzido por meio de partição em classes de equivalência, o que torna a quantidade de dados de entrada finita.

**023.** (CESPE/MEC/ANALISTA DE SISTEMAS/2011) Os testes de usabilidade avaliam a facilidade de uso do software testado e são bastante utilizados em aplicações web.

**024.** (CESPE/TRT 17 REGIÃO/ANALISTA JUDICIÁRIO/SISTEMAS/2013) O teste de carga, que verifica o funcionamento do software, utiliza uma grande quantidade de usuários simultaneamente.

**025.** (CESPE/TRE-BA/TÉCNICO JUDICIÁRIO/PROGRAMAÇÃO/2010) Teste funcional é uma técnica para se projetar casos de teste na qual o programa ou sistema é considerado uma caixa-preta e, para testá-lo, são fornecidas entradas e avaliadas as saídas geradas.

**026.** (CESPE/ANAC/ANALISTA ADMINISTRATIVO/TECNOLOGIA DA INFORMAÇÃO/2012) Os testes funcionais são caracterizados pelo uso do sistema conforme o seu usuário regular o faria.

**027.** (CESPE/ANATEL/ARQUITETURA DE SOLUÇÕES/2012) Considere as informações abaixo em relação ao desenvolvimento de sistemas:

I – Executar um software com o objetivo de revelar falhas, mas que não prova a exatidão do software.

II – Correta construção do produto.

III – Construção do produto certo.

Correspondem corretamente a I, II e III, respectivamente,

a) Validação, verificação e teste.

b) Verificação, teste e validação.

c) Teste, verificação e validação.

d) Validação, teste e verificação.

e) Teste, validação e verificação.

**028.** (FCC/TRT 14 REGIÃO/TÉCNICO JUDICIÁRIO/PROGRAMAÇÃO/2011) Garantir o funcionamento correto do software para atender as expectativas do cliente é o objetivo da homologação de sistemas. Nessa fase, que precede à implantação, os testes mais comuns são os testes:

a) Funcionais, de usabilidade e de aceitação.

b) De unidade, de iteração e de integração.

c) De volume, de integridade e de aceitação.

- d) Da caixa-branca, de carga e de configuração.
- e) De unidade, de carga e de integridade.

**029.** (IBFC/IMBEL/ESPECIALISTA/TECNOLOGIA DA INFORMAÇÃO/2024) Um teste de regressão em desenvolvimento de software, tem como missão:

- a) identificar defeitos no código-fonte
- b) verificar se as novas funcionalidades não interferem nas existentes
- c) testar a aplicação em diferentes ambientes
- d) avaliar o desempenho do sistema
- e) realizar testes de usabilidade

**030.** (CEBRASPE/CESPE/MPO/ANALISTA DE PLANEJAMENTO E ORÇAMENTO/2024) Julgue o item a seguir, relativos ao teste de estresse, a testes automatizados e à resiliência de aplicações.

Realiza-se o teste de estresse para confrontar os programas com situações anormais, de forma a exigir recursos em maior quantidade, frequência ou volume.

## GABARITO

- |              |              |              |
|--------------|--------------|--------------|
| <b>1.</b> C  | <b>11.</b> E | <b>21.</b> b |
| <b>2.</b> C  | <b>12.</b> C | <b>22.</b> C |
| <b>3.</b> C  | <b>13.</b> E | <b>23.</b> C |
| <b>4.</b> C  | <b>14.</b> c | <b>24.</b> C |
| <b>5.</b> E  | <b>15.</b> a | <b>25.</b> C |
| <b>6.</b> E  | <b>16.</b> e | <b>26.</b> E |
| <b>7.</b> a  | <b>17.</b> C | <b>27.</b> c |
| <b>8.</b> C  | <b>18.</b> c | <b>28.</b> a |
| <b>9.</b> C  | <b>19.</b> e | <b>29.</b> b |
| <b>10.</b> e | <b>20.</b> E | <b>30.</b> C |



## GABARITO COMENTADO

**003.** (QUADRIX/CFO/ANALISTA/DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escola da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

O teste de caixa branca concentra-se em testar a lógica interna do código e os fluxos de controle, garantindo que todas as possíveis condições e os caminhos do código sejam executados para verificar a cobertura de código e a precisão dos algoritmos.



A descrição apresentada corresponde precisamente à definição e aos objetivos do teste de caixa branca (*white-box testing*).

Os **testes de caixa-branca** buscam verificar o comportamento interno do software, ou seja, os elementos relacionados ao código-fonte desse software.

- **Foco na lógica interna:** o teste de caixa branca, também conhecido como teste estrutural, examina a estrutura interna do código, incluindo os fluxos de controle, as decisões lógicas e as condições;
- **Cobertura de código:** um dos principais objetivos do teste de caixa branca é garantir que todas as partes do código sejam executadas pelo menos uma vez, o que é medido pela cobertura de código;
- **Precisão dos algoritmos:** o teste de caixa branca também visa verificar se os algoritmos implementados no código estão funcionando corretamente e produzindo os resultados esperados.

Portanto, o item está certo. Ele descreve com precisão o foco e os objetivos do teste de caixa branca.

**Certo.**

**004.** (QUADRIX/CFO/ANALISTA/DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escola da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

Os testes de loops são importantes para verificar se os loops no código estão funcionando corretamente, testando tanto as condições de entrada quanto as de saída, e assegurando que o loop não caia em um loop infinito ou produza resultados incorretos.



A afirmação descreve a importância e os objetivos dos testes de *loops* de forma precisa.

- Verificação do Funcionamento Correto: testes de *loops* são essenciais para garantir que as estruturas de repetição no código (como *for*, *while*, *do-while*) executem o número correto de iterações e produzam os resultados esperados;
- Condições de Entrada e Saída: é crucial testar as condições que determinam quando o loop deve começar (condição de entrada) e quando deve terminar (condição de saída);
- Prevenção de Loops Infinitos: um dos principais objetivos é evitar que o *loop* entre em um ciclo infinito, o que pode causar o travamento do programa ou o consumo excessivo de recursos;
- Resultados Corretos: os testes de loops também devem verificar se as operações realizadas dentro do loop estão produzindo os resultados corretos em cada iteração.

Portanto, o item está certo. Ele destaca a importância dos testes de loops para garantir a qualidade e a confiabilidade do código.

**Certo.**

**005.** (CEBRASPE/CESPE/TRF 6ª REGIÃO/TÉCNICO JUDICIÁRIO/DESENVOLVIMENTO DE SISTEMAS DE INFORMAÇÃO/2025) A respeito de aplicação de testes, julgue o item a seguir. Para facilitar os testes de uma aplicação, podem ser utilizados os mock objects, que são objetos genéricos que atendem a todas as necessidades de testes.



*Mock Objects* são objetos simulados que imitam o comportamento de objetos reais, permitindo que os desenvolvedores testem o código de forma isolada, sem depender de outros componentes ou sistemas externos.

Trata-se de um recurso utilizado para simular o comportamento de um objeto real. Ele é conhecido, também, como um objeto simulado.

Ainda que os *mock objects* facilitem os testes de uma aplicação, a afirmação de que eles são objetos genéricos que atendem a todas as necessidades de testes é imprecisa e enganosa. *Mock objects* não são genéricos, mas sim específicos para um determinado cenário de teste. Eles são projetados para simular o comportamento de um objeto real em um contexto particular.

*Mock objects* são úteis para testes de unidade e integração, mas não substituem outros tipos de testes, como testes de sistema, testes de aceitação e testes de desempenho.

*Mock objects* são ferramentas valiosas para facilitar os testes, mas não são objetos genéricos que atendem a todas as necessidades de testes. Eles são específicos para cenários de teste particulares e complementam outros tipos de testes.

Portanto, o item está errado.

**Errado.**

**006.** (QUADRIX/CFO/ANALISTA/DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escolha da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

O teste de caixa preta é eficaz para verificar erros de implementação ou falhas lógicas no código, pois examina os processos internos do sistema e a estrutura do código.



A descrição apresentada refere-se ao teste de caixa branca, e não ao teste de caixa preta. O Teste “Caixa-Preta” (Black Box) analisa a funcionalidade e a aderência aos requisitos, em uma ótica externa ou do usuário, sem se basear em qualquer conhecimento do código e da lógica interna do componente testado. Concentra-se em testar a funcionalidade externa do software, sem conhecimento da sua estrutura interna ou do código fonte. Os testes são baseados nos requisitos e especificações do sistema, e o objetivo é verificar se o software atende às expectativas do usuário.

O Teste de Caixa Branca concentra-se em testar a lógica interna do código e os fluxos de controle, garantindo que todas as possíveis condições e os caminhos do código sejam executados para verificar a cobertura de código e a precisão dos algoritmos.

**Errado.**

**007.** (INQC/CPTRANS/PROGRAMADOR/2024) Quando se deseja apurar erros de interface durante os casos de teste de software, como por exemplo, o mau uso da interface, utiliza-se o teste:

- a) de componente
- b) de sistema
- c) de release
- d) unitário



A questão foca em apurar erros de interface durante os casos de teste. Embora o teste de sistema também envolva interfaces, o teste de componente é mais específico para avaliar a usabilidade da interface de cada componente individual do software.

- **Teste de componente:** avalia a usabilidade e o comportamento de cada componente individual, incluindo suas interfaces.

As outras opções não são adequadas:

- **Teste de sistema:** avalia o sistema como um todo, não se concentrando especificamente em erros de interface de componentes individuais;

- **Teste de release:** é uma etapa de testes em que uma equipe independente avalia a aplicação como um todo antes do lançamento;
- **Teste unitário:** concentra-se em testar pequenas partes do código (funções ou métodos) individualmente, não em testar a interface do componente.

Portanto, a resposta é a) de componente.

**Letra a.**

**008.** (CEBRASPE/CESPE/CAU BR/ANALISTA/INFRAESTRUTURA DE TECNOLOGIA DA INFORMAÇÃO/2024) Julgue o próximo item, relativo a testes de software.

Considerando que o teste funcional objetiva determinar se um recurso funciona corretamente sem problemas, é possível automatizar esse tipo de teste mesmo que o sistema seja web, em que é possível simular os retornos esperados.



O teste funcional verifica se cada função do software opera de acordo com a especificação de requisitos. Ele se concentra em validar se o sistema faz o que deveria fazer.

É possível automatizar testes funcionais em sistemas web utilizando ferramentas e *frameworks* como Selenium, Cypress, Playwright, etc. Essas ferramentas permitem simular as interações do usuário com a interface web, como cliques, preenchimento de formulários e navegação, e verificar se o comportamento do sistema é o esperado.

A automação de testes funcionais permite simular os retornos esperados do sistema, como a exibição de mensagens de sucesso, a validação de dados e a navegação para outras páginas. Isso garante que o teste seja executado de forma consistente e que os resultados sejam precisos.

É possível automatizar testes funcionais em sistemas web, simulando as interações do usuário e verificando se o comportamento do sistema é o esperado.

**Certo.**

**009.** (CEBRASPE/CESPE/CAU BR/ANALISTA/INFRAESTRUTURA DE TECNOLOGIA DA INFORMAÇÃO/2024) Julgue o próximo item, relativo a testes de software.

Considerada uma técnica sistemática para construir a arquitetura de software concomitantemente à realização de testes para descobrir erros associados às interfaces, o teste de integração realiza testes a partir de componentes testados em unidade.



O teste de integração, quando realizado de forma incremental (por exemplo, utilizando abordagens como “integração contínua”), pode influenciar a arquitetura do software, pois a identificação de problemas de integração pode levar a ajustes no *design* e na estrutura do sistema.

Um dos principais objetivos do teste de integração é verificar a correta interação entre os componentes do sistema, o que inclui a validação das interfaces entre esses componentes. O teste de integração é realizado após os componentes terem sido testados individualmente (teste de unidade). Isso garante que os componentes funcionem corretamente antes de serem integrados.

**Certo.**

---

**010.** (CESGRANRIO/IPEA/TÉCNICO DE PLANEJAMENTO E PESQUISA/DESENVOLVIMENTO DE SISTEMA/2024) Em projetos de desenvolvimento de softwares, é importante que eles sejam testados em diferentes situações comuns de ocorrerem, de modo a contribuir para que eles atinjam a qualidade esperada. Por isso, nesses projetos, quatro níveis de teste são importantes de serem considerados, a fim de ajudar a lidar com tais situações.

Esses quatro níveis de teste são os seguintes:

- a) acessibilidade, usabilidade, unidade e sistema
- b) desempenho, funcionais, componente e aceitação
- c) funcionais, não funcionais, componente e desempenho
- d) funcionais, não funcionais, usabilidade e acessibilidade
- e) unidade, integração, sistema e aceitação



Os quatro níveis de teste mais comuns e importantes no desenvolvimento de software, em ordem de granularidade e escopo, são:

- **Teste de Unidade:** foca em testar as menores unidades de código (funções, métodos, classes) de forma isolada;
- **Teste de Integração:** verifica a interação entre diferentes componentes ou módulos do sistema;
- **Teste de Sistema:** avalia o sistema como um todo, verificando se ele atende aos requisitos funcionais e não funcionais;
- **Teste de Aceitação:** realizado pelo cliente ou usuário final para validar se o sistema atende às suas necessidades e expectativas.

As outras opções apresentam combinações incorretas de tipos de teste ou incluem conceitos que não são considerados níveis de teste.

**Letra e.**

---

**011.** (CEBRASPE/CESPE/CAU BR/ANALISTA/SISTEMA DE INFORMAÇÃO/2024) Quanto aos princípios fundamentais das atividades de teste e aos processos ágeis de desenvolvimento de software, julgue o item a seguir.

Os estágios das atividades de teste de software devem ser realizados na seguinte ordem: teste do sistema, teste de integração e teste de unidade.



A ordem correta dos estágios de teste de software, seguindo uma abordagem de testes tradicional ou em processos ágeis, é a seguinte:

- Teste de Unidade: testes individuais de componentes ou módulos do software;
- Teste de Integração: testes que verificam a interação entre diferentes componentes ou módulos;
- Teste de Sistema: testes que avaliam o sistema como um todo, verificando se ele atende aos requisitos funcionais e não funcionais;
- Teste de Aceitação: testes realizados pelo cliente ou usuário final para validar se o sistema atende às suas necessidades e expectativas.

A ordem apresentada no item está invertida, indo do mais abrangente (sistema) para o mais granular (unidade).

Portanto, o item está errado.

**Errado.**

**012.** (CEBRASPE/CESPE/CAU BR/ANALISTA/SISTEMA DE INFORMAÇÃO/2024) Quanto aos princípios fundamentais das atividades de teste e aos processos ágeis de desenvolvimento de software, julgue o item a seguir.

Os testes de regressão são realizados por ocasião da ocorrência de mudanças no software.



Pressman (2021) destaca que *“o teste de regressão é a reexecução do mesmo subconjunto de testes que já foram executados, para assegurar que as alterações não tenham propagado efeitos colaterais indesejados. **Execute testes de regressão toda vez que for feita uma alteração grande no software** (incluindo a integração de novos componentes). **O teste de regressão ajuda a garantir que as alterações** (devido ao teste ou por outras razões) **não introduzam comportamento indesejado ou erros adicionais”**.*

**Certo.**

**013.** (CEBRASPE/CESPE/INPI/ANALISTA DE PLANEJAMENTO, GESTÃO E INFRAESTRUTURA EM PROPRIEDADE INDUSTRIAL/2024) Julgue o item que se segue, a respeito de qualidade de software.

Um teste de software de regressão estará corretamente projetado quando se considera, em cada uma das funções principais do software, apenas os testes que tratam de uma ou mais classes de erros.



O teste de regressão tem um objetivo mais amplo do que apenas testar classes específicas de erros. Ele visa garantir que as novas alterações ou correções no código não causem efeitos colaterais indesejados em funcionalidades já existentes.

- **Teste de Regressão:** é um tipo de teste que visa verificar se as alterações recentes no código (como correções de *bugs*, novas funcionalidades ou atualizações) não introduziram novos defeitos em áreas do sistema que já estavam funcionando corretamente;
- **Abrangência:** um bom conjunto de testes de regressão deve abranger as principais funcionalidades do sistema, e não apenas os testes relacionados a classes específicas de erros.

Um teste de regressão bem projetado deve incluir testes que verifiquem a funcionalidade das principais áreas do sistema, garantindo que as alterações recentes não tenham causado problemas em áreas já testadas e funcionando.

**Errado.**

---

**014.** (INSTITUTO AOCP/CÂMARA DE TERESINA-PI/ANALISTA DE INFORMÁTICA/2021) Cleiton está realizando testes em um sistema e revisa o código procurando por problemas. Nesse caso, ele está realizando testes de qual tipo?

- Unitário.
- Integração.
- Caixa Branca.
- Caixa Preta.
- A. B.



Se Cleiton está acessando código, portanto, **avalia a lógica interna do sistema!** Nesse contexto, está usando um teste de caixa branca.

**O teste “caixa-branca”** é uma abordagem para projetar casos de teste na qual os **testes são derivados do conhecimento da estrutura e da implementação do software**. O entendimento do algoritmo usado em um componente pode ajudar a identificar partições e casos de teste adicionais (SOMMERVILLE, 2007).

**O Teste “Caixa-Preta” (Black Box) analisa a funcionalidade e a aderência aos requisitos, em uma ótica externa ou do usuário, sem se basear em qualquer conhecimento do código e da lógica interna do componente testado.**



O **Técnica de Caixa Cinza** utiliza **conjuntamente as técnicas de caixa branca e caixa preta**, em que conhecemos a estrutura interna do software e verificamos as estradas e saídas das funcionalidades, exemplo disso é o teste de integração.

**Letra c.**

**015.** (INSTITUTO AOCP/PREFEITURA DE NOVO HAMBURGO-RS/TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS/2020) Qual das seguintes alternativas apresenta uma característica de um teste de caixa preta?

- a) Ignora os mecanismos internos de um sistema e focaliza as saídas geradas em resposta às entradas e condições de execução selecionadas.
- b) É um sinônimo de teste de caixa transparente.
- c) Leva em consideração os mecanismos internos de um sistema componente. É semelhante a um teste estrutural.
- d) Nesse tipo de teste, um exemplo de resultado é a quantidade de erros de acesso da aplicação ao banco de dados.
- e) Um teste de caixa preta é utilizado para determinar se todas as bibliotecas necessárias para a execução do sistema como um todo estão disponíveis para a aplicação.



O **Teste “Caixa-Preta” (Black Box, comportamental ou funcional)** analisa a funcionalidade e a aderência aos requisitos, em uma ótica externa ou do usuário, sem se basear em qualquer conhecimento do código e da lógica interna do componente testado. **Ele ignora os detalhes internos** e tem o objetivo de verificar a consistência entre o produto implementado e respectivos requisitos.

**Letra a.**

**016.** (INSTITUTO AOCP/PREFEITURA DE NOVO HAMBURGO-RS/TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS/2020) Assinale a alternativa que apresenta o objetivo de um teste funcional.

- a) Avaliar os aspectos de integridade do banco de dados.
- b) Verificar requisitos de desempenho, como o número de terminais suportados.
- c) Determinar os requisitos de guarda e retenção de dados.
- d) Mapear os aspectos de instabilidade do sistema.
- e) Verificar a consistência entre o produto implementado e respectivos requisitos.



O **teste caixa-preta (comportamental ou funcional)** ignora os mecanismos internos de um sistema e focaliza as saídas geradas em resposta às entradas e condições de execução

selecionadas. Verifica, portanto, a consistência entre o produto implementado e respectivos requisitos.

**Letra e.**

---

**017.** (CESPE/CEBRASPE/SERPRO/ANALISTA/ESPECIALIZAÇÃO DESENVOLVIMENTO DE SISTEMAS/2021) Acerca dos fundamentos e dos princípios da qualidade de software e da gestão da configuração, julgue o item que se segue.

Realizado o teste unitário de um módulo, o teste de integração contribuirá para a avaliação da existência de erros associados às interfaces do sistema.



O primeiro tipo de teste que temos é o **teste de unidade** que trabalha com técnicas de teste para detecção de erros em componentes menores (**unidades**) que serão integradas já testadas.

Em seguida temos o **teste de integração** que tem o objetivo de analisar problemas na construção (**integração**) dos componentes dentro de um software.

Em suma, o teste unitário testa o módulo e o de integração as interfaces!

**Certo.**

---

**018.** (CESPE/TJ-PA/ANALISTA JUDICIÁRIO/ANÁLISE DE SISTEMAS/2020) No processo de teste de software, o beta teste é

- a) realizado por uma equipe de teste independente.
- b) realizado pelos clientes no local de trabalho do desenvolvedor de software.
- c) realizado por clientes em seu próprio local de trabalho.
- d) útil para testar software sob medida.
- e) executado o mais cedo possível no ciclo de vida.



No **teste alfa** temos o usuário utilizando o software em **ambiente controlado** e no **teste beta** temos o usuário utilizando software em ambiente real.

Assim, no processo de teste de software, o beta teste é realizado por clientes em seu próprio local de trabalho.

**Letra c.**

---

**019.** (FGV/BANESTES/ANALISTA EM TECNOLOGIA DA INFORMAÇÃO/SUORTE E INFRAESTRUTURA/2018) No contexto de teste de software, o termo “Beta teste” caracteriza testes que:

- a) empregam primordialmente técnicas conhecidas como “White box”;
- b) são equivalentes aos testes conhecidos pelo termo “Alfa teste”;

- c) focam em pontos críticos, cujas correções são providenciadas de imediato pelos desenvolvedores;
- d) são realizados num ambiente de laboratório do desenvolvedor;
- e) são realizados por usuários externos, em condições de uso semelhantes às de produção.



No contexto de teste de software, o termo “Beta teste” caracteriza testes que são realizados por usuários externos, em condições de uso semelhantes às de produção.

No **teste alfa** temos o usuário utilizando o software em **ambiente controlado** e no **teste beta** temos o usuário utilizando software em ambiente real.

**Letra e.**

---

**020.** (CESPE/BNB/ESPECIALISTA TÉCNICO/ANALISTA DE SISTEMA/2018) O item a seguir apresenta uma situação hipotética, seguida de uma assertiva a ser julgada, com relação a testes de software.

Uma equipe de desenvolvimento de softwares pretendia realizar testes de forma incremental durante o desenvolvimento de um programa, a fim de verificar se mudanças no programa não haviam nele introduzido novos bugs; para isso, foram sugeridos os testes unitários e de regressão. Nessa situação, será correto utilizar os testes unitários, mas não os testes de regressão, pois esses últimos não visam verificar novos bugs, mas sim, tão somente, avaliar as funcionalidades do sistema.



O **teste de regressão** consiste em se aplicar, a cada nova versão do software ou a cada ciclo, **todos** os testes que já foram aplicados nas versões ou ciclos de teste anteriores do sistema para assegurar que as alterações não tenham propagado efeitos colaterais indesejados. Pode ser aplicado em qualquer nível de teste.

**Errado.**

---

**021.** (FGV/DPE-RJ/TÉCNICO SUPERIOR ESPECIALIZADO/DESENVOLVIMENTO DE SISTEMAS/2014) Testes unitários são amplamente empregados no desenvolvimento de software. Sua função principal é:

- a) testar o desempenho do software e de seus componentes.
- b) testar o menor bloco de software desenvolvido, avaliando os resultados obtidos com entradas de dados pré-definidos.
- c) avaliar o comportamento do sistema como um todo e como a integração dos componentes de software se comportam.

- d) avaliar a adequação do software desenvolvido com os requisitos unitários estruturais definidos pelos usuários.
- e) testar os cenários alternativos dos casos de uso, garantindo o comportamento esperado para o sistema.



Os **testes de unidade** (ou **testes unitários**) têm por objetivo explorar a menor unidade do projeto, procurando provocar falhas ocasionadas por defeitos de lógica e de implementação em cada módulo, separadamente. O universo alvo desse tipo de teste são os métodos dos objetos ou mesmo pequenos trechos de código.

**Letra b.**

---

**022.** (CESPE/MPE-PI/ANALISTA DE INFORMÁTICA/2012) Em teste funcional, o conjunto de valores de entrada válidos pode ser reduzido por meio de partição em classes de equivalência, o que torna a quantidade de dados de entrada finita.



Em um **teste funcional (caixa preta)** tem-se valores de entrada que podem ser reduzidos através de partição em classes, limitando assim a quantidade de dados.

**Certo.**

---

**023.** (CESPE/MEC/ANALISTA DE SISTEMAS/2011) Os testes de usabilidade avaliam a facilidade de uso do software testado e são bastante utilizados em aplicações web.



**Teste de usabilidade** é um tipo de teste utilizado para avaliar o uso do software.

**Certo.**

---

**024.** (CESPE/TRT 17 REGIÃO/ANALISTA JUDICIÁRIO/SISTEMAS/2013) O teste de carga, que verifica o funcionamento do software, utiliza uma grande quantidade de usuários simultaneamente.



No **teste de carga** verificamos o funcionamento do software, em situações extremas, como uma grande quantidade de usuários acessando uma funcionalidade ao mesmo tempo.

**Certo.**

---

**025.** (CESPE/TRE-BA/TÉCNICO JUDICIÁRIO/PROGRAMAÇÃO/2010) Teste funcional é uma técnica para se projetar casos de teste na qual o programa ou sistema é considerado uma caixa-preta e, para testá-lo, são fornecidas entradas e avaliadas as saídas geradas.



No teste caixa preta eu me preocupo com o que entra e o que sai do meu software, o que acontece no meio do caminho, na estrutura interna não é importante aqui, logo o teste caixa preta também é chamado de teste funcional, porque o importante aqui é o que as funcionalidades do software entregam na ponta, ou seja, quais são as saídas geradas.

Relembrando: A técnica de **caixa preta (funcional)** testa pré-condições avaliando o **comportamento das funcionalidades** conforme os requisitos **sem se preocupar com a lógica interna**.

**Certo.**

**026.** (CESPE/ANAC/ANALISTA ADMINISTRATIVO/TECNOLOGIA DA INFORMAÇÃO/2012) Os testes funcionais são caracterizados pelo uso do sistema conforme o seu usuário regular o faria.



Isso não faz o menor sentido quando estamos tratando de testes funcionais ou testes caixa preta, lembrando:

A técnica de **caixa preta (funcional)** testa pré-condições avaliando o **comportamento das funcionalidades** conforme os requisitos **sem se preocupar com a lógica interna**.

Não existe a necessidade de simular como o usuário faria, existe sim a necessidade de avaliar como as funcionalidades deveriam fazer.

**Errado.**

**027.** (CESPE/ANATEL/ARQUITETURA DE SOLUÇÕES/2012) Considere as informações abaixo em relação ao desenvolvimento de sistemas:

I – Executar um software com o objetivo de revelar falhas, mas que não prova a exatidão do software.

II – Correta construção do produto.

III – Construção do produto certo.

Correspondem corretamente a I, II e III, respectivamente,

a) Validação, verificação e teste.

b) Verificação, teste e validação.

c) Teste, verificação e validação.

d) Validação, teste e verificação.

e) Teste, validação e verificação.



Quando um software é executado com o objetivo de revelar falhas (comportamento não esperado) faz-se um **teste**, que obviamente não prova a exatidão do software como um todo.

**Verificação** é analisar se a forma com a qual estamos construindo um produto (software) é a correta, e **validação** é analisar se o produto (software) é o correto. As perguntas que diferenciam verificação de validação, são:

- Verificação: **será que estamos construindo o produto corretamente;**
- Validação: **será que estamos construindo o produto correto.**

Assim tem-se:

|                    |                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Teste</b>       | <b>I – Executar um software com o objetivo de revelar falhas, mas que não prova a exatidão do software.</b> |
| <b>Verificação</b> | <b>II – Correta construção do produto.</b>                                                                  |
| <b>Validação</b>   | <b>III – Construção do produto certo.</b>                                                                   |

**Letra c.**

**028.** (FCC/TRT 14 REGIÃO/TÉCNICO JUDICIÁRIO/PROGRAMAÇÃO/2011) Garantir o funcionamento correto do software para atender as expectativas do cliente é o objetivo da homologação de sistemas. Nessa fase, que precede à implantação, os testes mais comuns são os testes:

- Funcionais, de usabilidade e de aceitação.
- De unidade, de iteração e de integração.
- De volume, de integridade e de aceitação.
- Da caixa-branca, de carga e de configuração.
- De unidade, de carga e de integridade.



Para avaliar as expectativas do cliente vamos utilizar, na fase de homologação, os testes **funcionais** (analisamos se o que está saindo na ponta é de fato o que deveria estar saindo), teste de **usabilidade** (verificamos a utilização do software pelo usuário) e por fim o teste de **aceitação** (em que o usuário valida se o que foi apresentado enquanto software está em conformidade com os requisitos).

**Letra a.**

**029.** (IBFC/IMBEL/ESPECIALISTA/TECNOLOGIA DA INFORMAÇÃO/2024) Um teste de regressão em desenvolvimento de software, tem como missão:

- a) identificar defeitos no código-fonte
- b) verificar se as novas funcionalidades não interferem nas existentes
- c) testar a aplicação em diferentes ambientes
- d) avaliar o desempenho do sistema
- e) realizar testes de usabilidade



Quando utilizamos um teste de regressão em nossa aplicação, significa que estamos reexecutando um conjunto de testes feito anteriormente. A ideia precípua dos testes de regressão é verificar se as alterações feitas não afetaram o que já existia.

Segundo Pressmann (2021):

o teste de regressão é a reexecução do mesmo subconjunto de testes que já foram executados, para assegurar que as alterações não tenham propagado efeitos colaterais indesejados. Execute testes de regressão toda vez que for feita uma alteração grande no software (incluindo a integração de novos componentes). O teste de regressão ajuda a garantir que as alterações (devido ao teste ou por outras razões) não introduzam comportamento indesejado ou erros adicionais.

Assim, o principal objetivo do teste de regressão é garantir que as alterações recentes no código (como correções de *bugs*, novas funcionalidades ou atualizações) não introduzam novos defeitos em áreas do sistema que já estavam funcionando corretamente. Ele visa verificar se as funcionalidades existentes continuam operando como esperado após as modificações.

**Letra b.**

**030.** (CEBRASPE/CESPE/MPO/ANALISTA DE PLANEJAMENTO E ORÇAMENTO/2024) Julgue o item a seguir, relativos ao teste de estresse, a testes automatizados e à resiliência de aplicações.

Realiza-se o teste de estresse para confrontar os programas com situações anormais, de forma a exigir recursos em maior quantidade, frequência ou volume.



**Teste por esforço (stress):** é um tipo de teste de desempenho que visa avaliar a estabilidade e a capacidade de um sistema sob condições extremas ou anormais. Usa um sistema de maneira que **demande recursos** em quantidade, frequência ou volume **anormal**.

**Certo.**



## REFERÊNCIAS

CÉSAR, B. *Modelos de desenvolvimento de software: a Catedral e o Bazar*. Disponível em: <<http://www.brod.com.br/node/536>>. Acesso em: 11 jul. 2021.

CRISPIN, L.; GREGORY, J. *Agile testing condensed: a brief introduction*. Addison-Wesley Professional, 2018.

IBM. Disponível em: <[https://www.ibm.com/developerworks/community/blogs/ctaurion/entry/evolucao\\_do\\_open\\_source?lang=en](https://www.ibm.com/developerworks/community/blogs/ctaurion/entry/evolucao_do_open_source?lang=en)>. Acesso em: 11 jul. 2021.

IBM. *Teste de aceitação*. 1987. Disponível em: <[https://www.cin.ufpe.br/~gta/rup-vc/core.base\\_rup/guidances/concepts/acceptance\\_testing\\_12A0F152.html](https://www.cin.ufpe.br/~gta/rup-vc/core.base_rup/guidances/concepts/acceptance_testing_12A0F152.html)>. Acesso em: dez. de 2022.

KOSCIANSKI, A.; SOARES, M. S. *Qualidade de software*. São Paulo: Editora Novatec, 2007.

KRUCHTEN, P. *Introdução ao RUP – Rational Unified Process*. 2. ed. Rio de Janeiro: Ciência Moderna, 2003.

PFLEEGER, S. L. *Engenharia de software: teoria e prática*. 2.ed. São Paulo: Prentice Hall, 2004.

PRESSMAN, R. S. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.

QUINTÃO, P. L. *Notas de aula da disciplina “Tecnologia da Informação”*. 2025.

SOMMERVILLE, I. *Engenharia de software*. 8. ed. São Paulo: Pearson Addison-Wesley, 2007.

\_\_\_\_\_. *Engenharia de software*. 9. ed. São Paulo: Pearson Addison-Wesley, 2011.

Abra



caminhos



crie

futuros

gran.com.br

